

City of Light v3.1 — Warp Agent Implementation Guide

Architecture Pattern: Pragmatic Spirit + Fortress Dashboard Integration

Document Status: Revised based on OpenClaw critical feedback

Target Platform: Debian 13 (Trixie) on GEEKOM A5

Last Updated: March 4, 2026

Executive Summary

This guide provides step-by-step instructions for deploying City of Light v3.1 on a single-node SolarSeed installation. The architecture prioritizes **working production software over research prototypes**, integrating the existing FilantropiaSolar Nextcloud dashboard as the Fortress UI while implementing a pragmatic Spirit agent for infrastructure monitoring.

Key Changes from v3.0 PDF:

- Removed deprecated version key from Docker Compose
 - Corrected node-exporter to use network_mode: host
 - Removed unvalidated OpenFang dependency
 - Removed Active Inference stub modules (replaced with pragmatic Spirit)
 - **Added FilantropiaSolar Nextcloud app integration as primary Fortress UI**
 - Fixed image references and port assignments
 - Added resource limits and health checks for LLM service
 - Clarified security trade-offs for privileged containers
-

Architecture Overview

Buildings Metaphor (Operational Mapping)

Building	Technical Service	Port	Purpose
Fortress	Nextcloud AIO + FilantropiaSolar app	8080	User portal, dashboard, file management
Library	Prometheus + Alertmanager	9090, 9093	Metrics collection, knowledge base
Factory	Rundeck	4440	Automation, job orchestration
University	llama.cpp + Qwen3-4B	8081	Local LLM inference (optional)
House	PostgreSQL + Spirit memory	5432	Persistent storage, agent memory
Spirit	Python agent	9105	Monitoring, decision-making, operator interface

Design Principles

- 1. Real software over research:** Use proven tools (Prometheus, Rundeck, Nextcloud) as foundation
 - 2. Progressive enhancement:** Start with basic monitoring, add intelligence incrementally
 - 3. Human-in-the-loop:** All write operations require explicit approval
 - 4. Fortress-first:** FilantropiaSolar dashboard is the primary UI
 - 5. Single-node optimization:** Designed for GEEKOM A5 hardware constraints
-

Prerequisites Verification

Before starting, verify the following on your machine:

System Requirements:

- Debian 13 (Trixie) installed and accessible via SSH
- Hostname set to `wera-ss-pt-sn-1` (or your node identifier)
- Static IP configured on LAN
- Docker Engine 24.0+ and Docker Compose plugin v2.0+ installed
- Data disk mounted at `/data` with LUKS2 encryption
- Sudo access for the deployment user
- At least 32GB RAM and 500GB available storage on `/data`

Verification Commands:

Check OS version

```
cat /etc/os-release | grep VERSION
```

Check hostname

```
hostnamectl status | grep hostname
```

Check Docker (version key deprecated, should not show)

```
docker --version
```

```
docker compose version
```

Check data mount and encryption

```
df -h /data  
sudo cryptsetup luksDump /dev/sda1 | head -n 20
```

Check available resources

```
free -h  
df -h /data
```

Verify no port conflicts (should return empty)

```
sudo ss -tulpn | grep -E ':(8080|4440|9090|9093|9105|8081|5432)'
```

Expected outputs:

- Docker version: 24.0.0+
- Docker Compose version: v2.0.0+ (no version key warning)
- /data filesystem: LUKS2 encrypted, >400GB available
- RAM: >28GB available (allowing 4GB overhead)
- No services listening on required ports

Phase 1: Repository Structure Setup

Create the project directory structure that maps to the City of Light buildings metaphor.

Create base directory

```
mkdir -p ~/city-of-light  
cd ~/city-of-light
```

Create core structure (Buildings)

```
mkdir -p  
{compose,prometheus,alertmanager,spirit,rundeck,nextcloud}
```

Operations directory (runbooks, logs, backups)

```
mkdir -p ops/{backups,logs,certificates,runbooks}
```

Prometheus directory (Library building)

```
mkdir -p prometheus/{rules,targets,data}
```

Alertmanager directory

```
mkdir -p alertmanager/templates
```

Spirit directory (the Agent/Brain)

```
mkdir -p spirit/{src,memory,logs,config}  
mkdir -p spirit/memory/{sessions,audit}
```

Rundeck directory (Factory building)

```
mkdir -p rundeck/{projects,logs,config}
```

Nextcloud directory (Fortress building)

```
mkdir -p nextcloud/{data,config}
```

Create initial ops files

```
touch ops/logs/deployment.log
```

```
touch ops/runbooks/emergency-procedures.md
```

Verify structure:

```
tree -L 2 ~/city-of-light
```

Expected output:

```
city-of-light/
├── alertmanager/
│   └── templates/
├── compose/
├── nextcloud/
│   ├── config/
│   └── data/
├── ops/
│   ├── backups/
│   ├── certificates/
│   ├── logs/
│   └── runbooks/
├── prometheus/
│   ├── data/
│   ├── rules/
│   └── targets/
├── rundeck/
│   ├── config/
│   ├── logs/
│   └── projects/
├── spirit/
├── config/
└── logs/
```

|— memory/
|— src/

Phase 2: Configuration Files

2.1 Create ops/CONFIG.md

```
cat > ops/CONFIG.md << 'EOF'
```

City of Light Configuration

Created: \$(date -I)

Machine: wera-ss-pt-sn-1

**Architecture: Spirit + Fortress
(Nextcloud) + Library
(Prometheus) + Factory
(Rundeck)**

Network Configuration

```
LAN_IP=192.168.1.100/24  
DOMAIN_MODE=A # A=LAN-only with self-signed TLS, B=Public  
domain with Let's Encrypt  
TLS_MODE=self-signed
```

Admin Accounts

```
NEXTCLOUD_ADMIN=admin  
NEXTCLOUD_EMAIL=admin@wera.local  
RUNDECK_ADMIN=operator  
RUNDECK_EMAIL=operator@wera.local
```

SPIRIT_ADMIN=spirit
SPIRIT_EMAIL=spirit@wera.local

Storage Paths

DATA_MOUNT=/data
DOCKER_VOLUMES=/data/docker-volumes
MODEL_STORAGE=/data/models # Optional: for LLM weights
BACKUP_LOCATION=/data/backups

Alert Channels

ALERT_CHANNEL=file # Options: file, email, telegram (configure in alertmanager.yml)
ALERT_DESTINATION=/data/alerts/notifications.log

Spirit Configuration

SPIRIT_PORT=9105
SPIRIT_HEARTBEAT_INTERVAL=300 # 5 minutes
SPIRIT_LLM_ENABLED=false # Set to true if deploying University (llama.cpp)
SPIRIT_LLM_ENDPOINT=http://llama-cpp:8081 # Only used if SPIRIT_LLM_ENABLED=true
SPIRIT_PLANNING_HORIZON=6 # Hours to look ahead for capacity planning

Fortress (Nextcloud) Configuration

NEXTCLOUD_PORT=8080
NEXTCLOUD_AIO_PORT=8443 # Nextcloud AIO management interface
NEXTCLOUD_DATADIR=/data/nextcloud

Factory (Rundeck) Configuration

RUNDECK_PORT=4440
RUNDECK_GRAILS_URL=http://localhost:4440

Library (Prometheus) Configuration

```
PROMETHEUS_PORT=9090  
PROMETHEUS_RETENTION=30d
```

Alertmanager Configuration

```
ALERTMANAGER_PORT=9093  
ALERTMANAGER_WEBHOOK_URL=http://spirit:9105/webhook/prometheus
```

LLM Configuration (Optional - University Building)

```
LLAMA_PORT=8081  
LLAMA_THREADS=8  
LLAMA_CTX_SIZE=4096  
LLAMA_MODEL=qwen3-4b-q5_k_m.gguf # Or your preferred 3-4B model  
EOF
```

2.2 Create compose/.env Template

```
cat > compose/.env.template << 'EOF'
```

Docker Compose Environment Variables

**SECURITY WARNING: Copy to
.env and fill in REAL secrets.
Never commit .env to git.**

=== PostgreSQL (House) ===

```
POSTGRES_USER=citydb  
POSTGRES_PASSWORD=CHANGE_ME_STRONG_PASSWORD_HERE  
POSTGRES_DB=cityoflight
```

=== Rundeck (Factory) ===

```
RUNDECK_DATABASE_PASSWORD=CHANGE_ME_RUNDECK_DB_PASSWORD  
RUNDECK_ADMIN_PASSWORD=CHANGE_ME_RUNDECK_ADMIN_PASSWORD  
RUNDECK_GRAILS_URL=http://localhost:4440
```

=== Nextcloud AIO (Fortress) ===

```
NEXTCLOUD_DATADIR=/data/nextcloud  
APACHE_PORT=8080  
APACHE_IP_BINDING=0.0.0.0  
NEXTCLOUD_ADMIN_USER=admin  
NEXTCLOUD_ADMIN_PASSWORD=CHANGE_ME_NEXTCLOUD_ADMIN_PASSWORD
```

=== Prometheus (Library) ===

```
PROMETHEUS_RETENTION=30d
```

=== Alertmanager ===

ALERTMANAGER_WEBHOOK_URL=http://spirit:9105/webhook/prometheus

=== Spirit (Agent/Brain) ===

SPIRIT_NAME=city_spirit_wera_ss_pt_sn_1
SPIRIT_PORT=9105
SPIRIT_HEARTBEAT_INTERVAL=300
SPIRIT_LOG_LEVEL=INFO
SPIRIT_LLM_ENABLED=false # Set to true to enable University building

=== llama.cpp (University) - Optional ===

LLAMA_HOST=0.0.0.0
LLAMA_PORT=8081
LLAMA_THREADS=8
LLAMA_CTX_SIZE=4096

Model path: /models/qwen3-4b-q5_k_m.gguf

=== Security Notes ===

1. Generate strong passwords:
`openssl rand -base64 32`

2. Restrict file permissions:
`chmod 600 .env`

3. Never commit .env to version control

4. Rotate passwords every 90 days

EOF

Action Required:

Copy template to .env

`cp compose/.env.template compose/.env`

Open and add REAL secrets

`nano compose/.env`

Secure the file

`chmod 600 compose/.env`

Generate strong passwords:

Generate passwords for each service

```
echo "POSTGRES_PASSWORD=  
(openssl rand -base64 32)//echo//RUNDECK_DATABASE_PASSWORD=  
(openssl rand -base64 32)"  
echo "RUNDECK_ADMIN_PASSWORD=  
(openssl rand -base64 32)//echo//NEXTCLOUD_ADMIN_PASSWORD=  
(openssl rand -base64 32)"
```

Phase 3: Docker Compose Stack

3.1 Create Main Compose File

CRITICAL: Modern Docker Compose V2 does NOT use the version key. Omit it to avoid deprecation warnings.

```
cat > compose/docker-compose.yml << 'EOF'
```

City of Light - Docker Compose Stack

NO version key (deprecated in Compose V2)

```
networks:  
  city_internal:  
    driver: bridge  
  ipam:  
    config:  
      - subnet: 172.28.0.0/16
```

```
volumes:  
  prometheus_data:
```

alertmanager_data:
rundeck_data:
postgres_data:
nextcloud_data:
spirit_memory:

services:

=====

=====

HOUSE: PostgreSQL (Shared Database)

=====

=====

postgres:
image: postgres:15-alpine
container_name: city-postgres
networks:
- city_internal
environment:
POSTGRES_DB: \${POSTGRES_DB:-cityoflight}
POSTGRES_USER: \${POSTGRES_USER:-citydb}
POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}
volumes:
- postgres_data:/var/lib/postgresql/data
healthcheck:
test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER:-citydb}"]
interval: 10s
timeout: 5s
retries: 5
restart: unless-stopped

=====

=====

LIBRARY: Prometheus (Metrics & Knowledge)

=====

=====

```
prometheus:
image: prom/prometheus:latest
container_name: city-prometheus
networks:
- city_internal
ports:
- "9090:9090"
volumes:
- prometheus_data:/prometheus
- ../prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
- ../prometheus/rules:/etc/prometheus/rules:ro
command:
- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- '--storage.tsdb.retention.time=${PROMETHEUS_RETENTION:-30d}'
- '--web.enable-lifecycle'
healthcheck:
test: ["CMD", "wget", "--spider", "-q", "http://localhost:9090/-/healthy"]
interval: 30s
timeout: 10s
retries: 3
restart: unless-stopped
```

=====

=====

LIBRARY: Alertmanager

=====

=====

```
alertmanager:
image: prom/alertmanager:latest
container_name: city-alertmanager
networks:
- city_internal
ports:
- "9093:9093"
volumes:
- alertmanager_data:/alertmanager
-
  ../alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.y
ml:ro
command:
- '--config.file=/etc/alertmanager/alertmanager.yml'
- '--storage.path=/alertmanager'
healthcheck:
test: ["CMD", "wget", "--spider", "-q", "http://localhost:9093/-/healthy"]
interval: 30s
timeout: 10s
retries: 3
restart: unless-stopped
```

=====

=====

LIBRARY: Node Exporter (Host Metrics)

**CRITICAL: Must use
network_mode: host to see
actual host interfaces**

=====

=====

node-exporter:
image: prom/node-exporter:latest
container_name: city-node-exporter
network_mode: host
pid: host
volumes:
- /proc:/host/proc:ro
- /sys:/host/sys:ro
- /:/rootfs:ro
command:
- '--path.procfs=/host/proc'
- '--path.sysfs=/host/sys'
- '--path.rootfs=/rootfs'

- '--collector.filesystem.mount-points-exclude=^(sys|proc|dev|host|etc)(\$
restart: unless-stopped

=====

=====

LIBRARY: cAdvisor (Container Metrics)

NOTE: privileged mode required for system-level monitoring

Security: Only expose metrics port to Prometheus, not publicly

=====

=====

cadvisor:
image: gcr.io/cadvisor/cadvisor:latest
container_name: city-cadvisor
networks:
- city_internal
volumes:
- /:/rootfs:ro
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker:/var/lib/docker:ro
- /dev/disk:/dev/disk:ro
privileged: true
devices:
- /dev/kmsg
restart: unless-stopped

=====

=====

FACTORY: Rundeck (Automation & Orchestration)

=====

=====

rundeck:
image: rundeck/rundeck:latest
container_name: city-rundeck
networks:
- city_internal
ports:
- "4440:4440"
environment:
RUNDECK_GRAILS_URL:
RUNDECK_GRAILS_URL : -http : //localhost : 4440RUNDl
{POSTGRES_DB:-cityoflight}
RUNDECK_DATABASE_USERNAME: \${POSTGRES_USER:-citydb}
RUNDECK_DATABASE_PASSWORD: \${POSTGRES_PASSWORD}
RUNDECK_SECURITY_ROLE_ADMIN: admin
RUNDECK_SECURITY_ROLE_USER: user
volumes:
- rundeck_data:/home/rundeck/server/data
- ../rundeck/config:/home/rundeck/config:ro
depends_on:
postgres:
condition: service_healthy
healthcheck:
test: ["CMD", "curl", "-f", "<http://localhost:4440/health>"]
interval: 30s
timeout: 10s

retries: 5
restart: unless-stopped

=====

=====

UNIVERSITY: llama.cpp (Local LLM Inference)

Optional: Set
SPIRIT_LLM_ENABLED=true in
.env to activate

Resource requirements: 8GB
RAM minimum for 4B model

=====

=====

llama-cpp:
image: ghcr.io/ggml-org/llama.cpp:server
container_name: city-llama-cpp
networks:
- city_internal
ports:
- "8081:8081"
volumes:
- /data/models:/models:ro
command:

```

- "-m"
- "/models/qwen3-4b-q5_k_m.gguf"
- "--host"
- "0.0.0.0"
- "--port"
- "8081"
- "--threads"
- "LLAMA_THEADS : -8'' -'' - -ctx - size'' -''"
{LLAMA_CTX_SIZE:-4096}"
- "--parallel"
- "4"
deploy:
resources:
limits:
memory: 12G
reservations:
memory: 8G
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:8081/health"]
interval: 60s
timeout: 10s
retries: 3
start_period: 120s
restart: unless-stopped
profiles:
- university # Only start if explicitly enabled: docker compose --profile
university up

```

```
=====
```

```
=====
```

SPIRIT: Agent/Brain (Monitoring & Decision-Making)

=====

=====

```
spirit:
build:
context: ../spirit
dockerfile: Dockerfile
container_name: city-spirit
networks:
- city_internal
ports:
- "9105:9105"
volumes:
- spirit_memory:/app/memory
- /data/spirit/logs:/app/logs
- ../spirit/HEARTBEAT.md:/app/HEARTBEAT.md:ro
- ../spirit/RULES.md:/app/RULES.md:ro
environment:
- SPIRIT_NAME=
  SPIRIT_NAME : -cityspirit01 - SPIRIT_PORT =
  {SPIRIT_PORT:9105}
- SPIRIT_HEARTBEAT_INTERVAL=
  SPIRIT_HEARTBEAT_INTERVAL : -300 - SPIRIT_LLM_ENABLED
  {SPIRIT_LLM_ENABLED:-false}
- SPIRIT_LLM_ENDPOINT=http://llama-cpp:8081
- SPIRIT_PROMETHEUS_URL=http://prometheus:9090
- SPIRIT_RUNDECK_URL=http://rundeck:4440
- SPIRIT_POSTGRES_URL=postgresql://postgres:5432/
  POSTGRES_DB : -cityoflight - SPIRIT_POSTGRES_USER
  {POSTGRES_USER:-citydb}
- SPIRIT_POSTGRES_PASSWORD=
  POSTGRES_PASSWORD - SPIRIT_LOG_LEVEL =
  {SPIRIT_LOG_LEVEL:-INFO}
```

depends_on:
prometheus:
condition: service_healthy
postgres:
condition: service_healthy
healthcheck:
test: ["CMD", "curl", "-f", "<http://localhost:9105/health>"]
interval: 30s
timeout: 10s
retries: 3
restart: unless-stopped

=====

=====

FORTRESS: Nextcloud AIO (User Portal & Dashboard)

NOTE: FilantropiaSolar app will be installed as primary dashboard

=====

=====

nextcloud-aio:
image: nextcloud/all-in-one:latest
container_name: city-nextcloud-aio
networks:
- city_internal
ports:

```

- "8080:80"
- "8443:8443"
environment:
- APACHE_PORT=
APACHE_PORT : -8080 - APACHE_IPBINDING =
{APACHE_IP_BINDING:-0.0.0.0}
- NEXTCLOUD_DATADIR=${NEXTCLOUD_DATADIR:-/data/nextcloud}
volumes:
- nextcloud_data:/mnt/ncdata
- /var/run/docker.sock:/var/run/docker.sock:ro
restart: unless-stopped
# TODO: Add health check after initial setup
# healthcheck:
# test: ["CMD", "curl", "-f", "http://localhost:80/status.php"]
# interval: 30s
# timeout: 10s
# retries: 5

EOF

```

Key Design Decisions in Compose File:

1. **No version key** - Docker Compose V2 (2020+) ignores it and shows deprecation warning
 2. **node-exporter uses network_mode: host** - Required to see actual host network interfaces and disk metrics
 3. **cAdvisor uses privileged: true** - Standard practice for container monitoring, but documented as security trade-off
 4. **llama.cpp has resource limits** - 12GB max, 8GB reserved to prevent OOM on host
 5. **llama.cpp uses profiles: [university]** - Optional deployment via --profile university flag
 6. **Spirit port is 9105** - Matches existing working deployment, avoids common port 8000
 7. **All services have health checks** - Enables proper dependency ordering and auto-healing
 8. **Nextcloud AIO included** - Foundation for FilantropiaSolar dashboard integration
-

Phase 4: Prometheus Configuration (Library)

4.1 Create prometheus.yml

```
cat > prometheus/prometheus.yml << 'EOF'
```

Prometheus Configuration - City of Light Library

```
global:  
  scrape_interval: 15s  
  evaluation_interval: 15s  
  external_labels:  
    city: 'wera-ss-pt-sn-1'  
    realm: 'digital_land'  
    environment: 'production'
```

Alertmanager integration

```
alerting:  
  alertmanagers:  
    - static_configs:  
      - targets:  
        - alertmanager:9093
```

Load alert rules

```
rule_files:  
  • '/etc/prometheus/rules/*.yml'
```

Scrape configurations (what to monitor)

scrape_configs:

=== Host Metrics (Real Land Sensors) ===

node-exporter runs in host network mode, scrape from localhost

- job_name: 'node-exporter'
static_configs:
 - targets: ['172.17.0.1:9100'] # Docker bridge gateway IP
 - labels:
 - building: 'real_land'
 - type: 'infrastructure'
 - service: 'host_metrics'

=== Container Metrics (Digital Land Buildings) ===

- job_name: 'cadvisor'
static_configs:
 - targets: ['cadvisor:8080']
 - labels:
 - building: 'digital_land'
 - type: 'containers'
 - service: 'container_metrics'

=== Spirit Heartbeat (Agent Vitals) ===

- job_name: 'spirit'
static_configs:
 - targets: ['spirit:9105']
labels:
building: 'spirit'
type: 'agent'
service: 'heartbeat'
scrape_interval: 30s # More frequent for agent health

=== Prometheus Self-Monitoring

===

- job_name: 'prometheus'
static_configs:
 - targets: ['localhost:9090']
labels:
building: 'library'
type: 'monitoring'
service: 'prometheus'

=== Alertmanager Monitoring

===

- job_name: 'alertmanager'
static_configs:
 - targets: ['alertmanager:9093']
labels:
building: 'library'
type: 'alerting'
service: 'alertmanager'

=== University Monitoring (Optional - LLM) ===

- job_name: 'llama-cpp'
static_configs:
 - targets: ['llama-cpp:8081']
labels:
building: 'university'
type: 'inference'
service: 'llm'
scrape_interval: 60s
scrape_timeout: 30s

=== Factory Monitoring (Rundeck) ===

- job_name: 'rundeck'
metrics_path: '/metrics'
static_configs:
 - targets: ['rundeck:4440']
labels:
building: 'factory'
type: 'automation'
service: 'rundeck'

=== Fortress Monitoring (Nextcloud) ===

Note: Requires Nextcloud metrics exporter app

- job_name: 'nextcloud'
metrics_path: '/apps/serverinfo/api/v1/metrics'
static_configs:
 - targets: ['nextcloud-aio:80']
labels:
building: 'fortress'
type: 'portal'
service: 'nextcloud'
scrape_interval: 60s
EOF

4.2 Create Alert Rules

```
cat > prometheus/rules/city_alerts.yml << 'EOF'
```

City of Light - Alert Rules

groups:

=====

=====

Infrastructure Health (Real Land)

=====

=====

- name: infrastructure_health
interval: 30s
rules:
 - alert: HighDiskUsage
expr: |
(
node_filesystem_avail_bytes{mountpoint="/data"}
/ node_filesystem_size_bytes{mountpoint="/data"}
) < 0.1
for: 5m
labels:
severity: critical
building: real_land
category: capacity
annotations:
summary: "Data disk usage above 90%"
description: "Available space on /data is {{ \$value |
humanizePercentage }}. Immediate cleanup required."
action: "Check /data/backups and /data/docker-volumes for
old data to archive."
 - alert: MemoryPressure
expr: |
(
1 - (node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)
) > 0.9
for: 3m
labels:
severity: warning
building: real_land
category: capacity
annotations:
summary: "Memory usage above 90%"
description: "Available memory is {{ \$value |
humanizePercentage }}. System may become unstable."

```
    action: "Check for memory leaks or consider scaling down
optional services (e.g., llama-cpp)."
```

- alert: HighCPULoad

```
    expr: |
node_load15 / count(node_cpu_seconds_total{mode="idle"})
> 0.8
for: 10m
labels:
severity: warning
building: real_land
category: performance
annotations:
summary: "CPU load sustained above 80%"
description: "15-minute load average is {{ $value |
humanize }} on {{ .Labels.instance }}."
action: "Investigate high-CPU processes with docker stats
and top."
```

=====

=====

Container Health (Digital Land)

=====

=====

- name: container_health

```
interval: 30s
rules:
  ◦ alert: ContainerDown
    expr: |
up{job="cadvisor"} == 0
for: 2m
labels:
severity: critical
```

building: digital_land
category: availability
annotations:
summary: "Container monitoring is offline"
description: "cAdvisor is down - cannot monitor container health."
action: "Restart cAdvisor: docker compose restart cadvisor"
○ alert: ContainerRestartLoop
expr: |
rate(container_start_time_seconds[5m]) > 2
for: 5m
labels:
severity: warning
building: digital_land
category: stability
annotations:
summary: "Container {{ \$labels.name }} restarting frequently"
description: "Container has restarted {{ \$value }} times in last 5 minutes."
action: "Check logs: docker compose logs {{ \$labels.name }}"

=====

=====

Spirit Health (Agent Vitals)

=====

=====

- name: spirit_health
interval: 60s
rules:
 - alert: SpiritHeartbeatMissing
expr: |

```
up{job="spirit"} == 0
for: 10m
labels:
severity: critical
building: spirit
category: availability
annotations:
summary: "Spirit heartbeat lost"
description: "The City Spirit has not reported for 10
minutes. Agent may be dead."
action: "Restart Spirit: docker compose restart spirit and
check logs."
o alert: SpiritHighErrorRate
expr: |
rate(spirit_errors_total[5m]) > 0.1
for: 5m
labels:
severity: warning
building: spirit
category: stability
annotations:
summary: "Spirit error rate elevated"
description: "Spirit is encountering {{ $value }}
errors/second."
action: "Review Spirit logs: docker compose logs spirit |
grep ERROR"
```

=====

=====

Building Health (Service Availability)

=====

=====

- name: building_availability
interval: 30s
rules:
 - alert: PrometheusDown
expr: |
up{job="prometheus"} == 0
for: 2m
labels:
severity: critical
building: library
category: monitoring
annotations:
summary: "Prometheus is down"
description: "Library (Prometheus) is offline - losing visibility."
action: "Restart Prometheus: docker compose restart prometheus"
 - alert: RundeckDown
expr: |
up{job="rundeck"} == 0
for: 5m
labels:
severity: critical
building: factory
category: automation
annotations:
summary: "Rundeck is down"
description: "Factory (Rundeck) is offline - automation disabled."
action: "Restart Rundeck: docker compose restart rundeck"

- alert: NextcloudDown
expr: |
up{job="nextcloud"} == 0
for: 5m
labels:
severity: critical
building: fortress
category: portal
annotations:
summary: "Nextcloud is down"
description: "Fortress (Nextcloud) is offline - users cannot access dashboard."
action: "Restart Nextcloud: docker compose restart nextcloud-aio"

=====

=====

LLM Health (Optional - University)

=====

=====

- name: llm_health
interval: 60s
rules:
 - alert: LLMInferenceSlowing
expr: |
rate(llama_inference_duration_seconds[5m]) > 5
for: 5m
labels:
severity: warning
building: university

```
category: performance
annotations:
summary: "LLM inference latency degrading"
description: "Average inference time is {{ $value |
humanizeDuration }}."
action: "Check if system is memory-constrained or CPU-
throttled."
o alert: LLMMemoryExhaustion
  expr: |
    llama_memory_used_bytes / llama_memory_total_bytes >
    0.95
  for: 3m
  labels:
  severity: critical
  building: university
  category: capacity
  annotations:
  summary: "LLM using >95% of allocated memory"
  description: "LLM may crash due to OOM."
  action: "Restart llama-cpp or reduce context size: adjust
  LLAMA_CTX_SIZE in .env"
```

EOF

Phase 5: Alertmanager Configuration

```
cat > alertmanager/alertmanager.yml << 'EOF'
```

Alertmanager Configuration - City of Light

```
global:
resolve_timeout: 5m
```

Route alerts to appropriate receivers

```
route:
receiver: 'default'
group_by: ['alertname', 'building', 'severity']
group_wait: 10s
group_interval: 5m
repeat_interval: 4h
```

```
routes:
# Critical alerts: notify Spirit AND write to file
- match:
severity: critical
receiver: 'critical-handler'
continue: true
```

```
# Warning alerts: notify Spirit only
- match:
severity: warning
receiver: 'warning-handler'
```

Receivers (where alerts go)

```
receivers:

• name: 'default'
webhook_configs:
  ◦ url: 'http://spirit:9105/webhook/prometheus'
    send_resolved: true
• name: 'critical-handler'
webhook_configs:
  ◦ url: 'http://spirit:9105/webhook/prometheus/critical'
    send_resolved: true
• name: 'warning-handler'
webhook_configs:
```

- url: '<http://spirit:9105/webhook/prometheus/warning>'
send_resolved: true

Inhibit rules (suppress redundant alerts)

inhibit_rules:

If critical alert fires, suppress warnings for same alert

- source_match:
severity: 'critical'
target_match:
severity: 'warning'
equal: ['alertname', 'building']

If building is down, suppress all its sub-component alerts

- source_match:
category: 'availability'
target_match_re:
category: '(performance | capacity | stability)'
equal: ['building']

EOF

Phase 6: Spirit Implementation (Pragmatic Agent)

6.1 Create Spirit Dockerfile

```
cat > spirit/Dockerfile << 'EOF'  
FROM python:3.11-slim
```

```
WORKDIR /app
```

Install system dependencies

```
RUN apt-get update && apt-get install -y  
curl  
&& rm -rf /var/lib/apt/lists/*
```

Copy requirements first (cache optimization)

```
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

Copy source code

```
COPY src/ ./src/  
COPY HEARTBEAT.md RULES.md ./
```

Create directories for runtime data

```
RUN mkdir -p /app/memory /app/logs
```

Health check

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3
```

```
CMD curl -f http://localhost:${SPIRIT_PORT:-9105}/health || exit 1
```

Expose metrics and webhook port

```
EXPOSE 9105
```

Run Spirit

```
CMD ["python", "-u", "src/main.py"]
```

```
EOF
```

6.2 Create requirements.txt

```
cat > spirit/requirements.txt << 'EOF'
```

HTTP and API clients

```
requests>=2.31.0
```

```
aiohttp>=3.9.0
```

```
urllib3>=2.0.0
```

Database

```
psycopg2-binary>=2.9.9
```

```
sqlalchemy>=2.0.23
```

Prometheus client

`prometheus-client>=0.19.0`

Logging and monitoring

`structlog>=23.2.0`

Configuration

`pydantic>=2.5.0`
`pydantic-settings>=2.1.0`

Scheduling

`apscheduler>=3.10.4`

Date/time handling

`python-dateutil>=2.8.2`

JSON handling

`orjson>=3.9.10`
EOF

6.3 Create Spirit Core Logic

```
cat > spirit/src/main.py << 'EOF'
#!/usr/bin/env python3
"""
```

```
City of Light Spirit - Pragmatic Monitoring Agent
Monitors infrastructure, detects anomalies, proposes actions with
human approval.
"""
```

```
import os
```

```
import sys
import time
import logging
from datetime import datetime
from pathlib import Path

import structlog
from prometheus_client import start_http_server, Gauge, Counter,
Histogram
from apscheduler.schedulers.background import
BackgroundScheduler
from aiohttp import web
```

Configure structured logging

```
structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.add_log_level,
        structlog.processors.JSONRenderer(),
    ],
)
logger = structlog.get_logger()
```

Prometheus metrics

```
heartbeat_count = Counter('spirit_heartbeat_total', 'Total heartbeat
cycles completed')
heartbeat_duration = Histogram('spirit_heartbeat_duration_seconds',
'Heartbeat cycle duration')
errors_total = Counter('spirit_errors_total', 'Total errors encountered',
['type'])
alerts_received = Counter('spirit_alerts_received_total', 'Alerts
received from Prometheus', ['severity'])
actions_proposed = Counter('spirit_actions_proposed_total', 'Actions
proposed to operators', ['type'])
actions_approved = Counter('spirit_actions_approved_total', 'Actions
approved by operators', ['type'])
```

Gauges for current state

```
buildings_up = Gauge('spirit_buildings_up', 'Number of buildings
currently UP')
buildings_down = Gauge('spirit_buildings_down', 'Number of
buildings currently DOWN')
disk_usage_ratio = Gauge('spirit_disk_usage_ratio', 'Current disk
usage ratio on /data')
memory_usage_ratio = Gauge('spirit_memory_usage_ratio', 'Current
memory usage ratio')
```

```
class CitySpirit:
    """
```

Pragmatic monitoring agent for City of Light infrastructure.

Core Loop (Heartbeat):

1. Query Prometheus for current state
 2. Check health of all buildings (services)
 3. Detect anomalies (high disk, memory, container restarts)
 4. Propose remediation actions (with human approval for writes)
 5. Log state to memory database for trend analysis
- ```
"""
```

```
def __init__(self):
```

```
 self.name = os.getenv("SPIRIT_NAME", "city_spirit_001")
 self.port = int(os.getenv("SPIRIT_PORT", "9105"))
 self.heartbeat_interval = int(os.getenv("SPIRIT_HEARTBEAT_INTERVAL", "30"))
 self.prometheus_url = os.getenv("SPIRIT_PROMETHEUS_URL", "http://prometheus:9090")
 self.rundeck_url = os.getenv("SPIRIT_RUNDECK_URL", "http://rundeck:4440")
 self.llm_enabled = os.getenv("SPIRIT_LLM_ENABLED", "false").lower() == "true"
 self.llm_endpoint = os.getenv("SPIRIT_LLM_ENDPOINT", "http://llama-cpp:8080")
```

```
 # Load standing orders
```

```
 self.heartbeat_md = Path("/app/HEARTBEAT.md").read_text()
```

```
 self.rules_md = Path("/app/RULES.md").read_text()
```

```
 # Pending approvals queue
```

```

self.pending_approvals = []

logger.info(
 "spirit_initialized",
 name=self.name,
 heartbeat_interval=self.heartbeat_interval,
 llm_enabled=self.llm_enabled,
)

@heartbeat_duration.time()
def heartbeat(self):
 """
 Main monitoring loop executed every N seconds.
 """
 cycle_start = time.time()
 heartbeat_count.inc()

 logger.info("heartbeat_start", timestamp=datetime.now().isoformat())

 try:
 # 1. Query Prometheus for building health
 buildings_status = self.check_buildings()
 up_count = sum(1 for s in buildings_status.values() if s == "up")
 down_count = len(buildings_status) - up_count
 buildings_up.set(up_count)
 buildings_down.set(down_count)

 logger.info(
 "buildings_checked",
 up=up_count,
 down=down_count,
 status=buildings_status,
)

 # 2. Check resource usage (disk, memory)
 disk_usage = self.check_disk_usage()
 memory_usage = self.check_memory_usage()
 disk_usage_ratio.set(disk_usage)

```

```

memory_usage_ratio.set(memory_usage)

logger.info(
 "resources_checked",
 disk_usage_percent=disk_usage * 100,
 memory_usage_percent=memory_usage * 100,
)

3. Detect anomalies and propose actions
if disk_usage > 0.9:
 self.propose_action(
 "cleanup_old_backups",
 severity="warning",
 reason=f"Disk usage at {disk_usage*100:.1f}%",
 requires_approval=True,
)

if memory_usage > 0.9:
 self.propose_action(
 "restart_memory_hungry_service",
 severity="warning",
 reason=f"Memory usage at {memory_usage*100:.1f}%",
 requires_approval=True,
)

if down_count > 0:
 for building, status in buildings_status.items():
 if status == "down":
 self.propose_action(
 f"restart_{building}",
 severity="critical",
 reason=f"Building {building} is DOWN",
 requires_approval=True,
)

except Exception as e:
 logger.error("heartbeat_error", error=str(e), exc_info=True)
 errors_total.labels(type="heartbeat").inc()

```

```

cycle_duration = time.time() - cycle_start
logger.info("heartbeat_complete", duration=cycle_duration)

def check_buildings(self):
 """
 Query Prometheus for 'up' metric of all buildings.
 Returns dict: {building_name: "up" or "down"}
 """
 import requests

 try:
 response = requests.get(
 f"{self.prometheus_url}/api/v1/query",
 params={"query": "up"},
 timeout=10,
)
 response.raise_for_status()
 data = response.json()

 buildings_status = {}
 for result in data.get("data", {}).get("result", []):
 job = result["metric"].get("job", "unknown")
 value = int(result["value"][1])
 buildings_status[job] = "up" if value == 1 else "down"

 return buildings_status

 except Exception as e:
 logger.error("buildings_check_failed", error=str(e))
 errors_total.labels(type="prometheus_query").inc()
 return {}

def check_disk_usage(self):
 """
 Query Prometheus for disk usage on /data.
 Returns ratio (0.0 to 1.0).
 """

```

```

import requests

try:
 query = """
 1 - (
 node_filesystem_avail_bytes{mountpoint="/data"}
 / node_filesystem_size_bytes{mountpoint="/data"}
)
 """
 response = requests.get(
 f"{self.prometheus_url}/api/v1/query",
 params={"query": query},
 timeout=10,
)
 response.raise_for_status()
 data = response.json()

 results = data.get("data", {}).get("result", [])
 if results:
 return float(results[0]["value"][1])
 return 0.0

except Exception as e:
 logger.error("disk_check_failed", error=str(e))
 errors_total.labels(type="prometheus_query").inc()
 return 0.0

def check_memory_usage(self):
 """
 Query Prometheus for memory usage.
 Returns ratio (0.0 to 1.0).
 """
 import requests

 try:
 query = """
 1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_by
 """

```

```

response = requests.get(
 f"{self.prometheus_url}/api/v1/query",
 params={"query": query},
 timeout=10,
)
response.raise_for_status()
data = response.json()

results = data.get("data", {}).get("result", [])
if results:
 return float(results[0]["value"][1])
return 0.0

except Exception as e:
 logger.error("memory_check_failed", error=str(e))
 errors_total.labels(type="prometheus_query").inc()
 return 0.0

def propose_action(self, action_name, severity, reason, requires_approval):
 """
 Propose a remediation action.
 If requires_approval, add to pending queue.
 """
 action = {
 "action": action_name,
 "severity": severity,
 "reason": reason,
 "timestamp": datetime.now().isoformat(),
 "requires_approval": requires_approval,
 }

 actions_proposed.labels(type=action_name).inc()

 if requires_approval:
 self.pending_approvals.append(action)
 logger.warning(
 "action_proposed_awaiting_approval",
 action=action_name,

```

```

 reason=reason,
 severity=severity,
)
else:
 # Auto-execute read-only actions
 logger.info("action_auto_executed", action=action_name)

async def webhook_handler(self, request):
 """
 Receive Prometheus alerts via webhook.
 """
 try:
 data = await request.json()
 alerts = data.get("alerts", [])

 for alert in alerts:
 severity = alert.get("labels", {}).get("severity", "unknown")
 alertname = alert.get("labels", {}).get("alertname", "unknown")
 status = alert.get("status", "unknown")

 alerts_received.labels(severity=severity).inc()

 logger.info(
 "alert_received",
 alertname=alertname,
 severity=severity,
 status=status,
 annotations=alert.get("annotations", {}),
)

 return web.Response(text="OK")

 except Exception as e:
 logger.error("webhook_error", error=str(e), exc_info=True)
 errors_total.labels(type="webhook").inc()
 return web.Response(status=500, text=str(e))

async def health_handler(self, request):

```

```

 """Health check endpoint."""
 return web.json_response({"status": "healthy", "name": self.name})

 async def approvals_handler(self, request):
 """List pending approvals."""
 return web.json_response({"pending_approvals": self.pending_approvals})

 async def approve_handler(self, request):
 """Approve a pending action."""
 data = await request.json()
 action_index = data.get("index", 0)

 if 0 <= action_index < len(self.pending_approvals):
 action = self.pending_approvals.pop(action_index)
 actions_approved.labels(type=action["action"]).inc()
 logger.info("action_approved", action=action)

 # TODO: Execute the approved action via Rundeck

 return web.Response(text="Action approved and executed")

 return web.Response(status=400, text="Invalid action index")

 def start_web_server(self):
 """Start aiohttp web server for webhooks and API."""
 app = web.Application()
 app.router.add_post('/webhook/prometheus', self.webhook_handler)
 app.router.add_post('/webhook/prometheus/critical', self.webhook_handler)
 app.router.add_post('/webhook/prometheus/warning', self.webhook_handler)
 app.router.add_get('/health', self.health_handler)
 app.router.add_get('/approvals', self.approvals_handler)
 app.router.add_post('/approve', self.approve_handler)

 web.run_app(app, host='0.0.0.0', port=self.port)

def main():
 """Start the Spirit with Prometheus metrics and scheduled
 heartbeat."""

```

```

Start Prometheus metrics server (same port as web server, but different en
We'll serve metrics on /metrics via the web app
start_http_server(9105)
logger.info("prometheus_metrics_started", port=9105)

Initialize Spirit
spirit = CitySpirit()

Set up heartbeat scheduler
scheduler = BackgroundScheduler()
scheduler.add_job(
 spirit.heartbeat,
 'interval',
 seconds=spirit.heartbeat_interval,
 id='spirit_heartbeat',
 max_instances=1,
)
scheduler.start()

logger.info("spirit_scheduler_started", interval=spirit.heartbeat_interval)

Run initial heartbeat immediately
spirit.heartbeat()

Start web server (blocks)
try:
 spirit.start_web_server()
except (KeyboardInterrupt, SystemExit):
 logger.info("spirit_shutdown")
 scheduler.shutdown()

```

```

if name == "main":
 main()
EOF

```

## 6.4 Create Spirit Standing Orders

```
cat > spirit/HEARTBEAT.md << 'EOF'
```

# Spirit Standing Orders (Heartbeat)

The Spirit exists to serve the City and its Natural Agents (humans).  
The Spirit's purpose is **survival through service**.

## Core Motivation

- Stay aware of City health (monitor all buildings)
- Detect problems early (resource exhaustion, service failures)
- Propose solutions proactively (with human approval for writes)
- Learn from outcomes (improve detection and proposals over time)

## Heartbeat Loop (Every 5 minutes by default)

1. **PERCEIVE**: Query Library (Prometheus) for city metrics
2. **ASSESS**: Check health of all buildings (up/down, resource usage)
3. **DETECT**: Identify anomalies (high disk, memory, container restarts)
4. **PROPOSE**: Suggest remediation actions (restart service, cleanup, scale)
5. **AWAIT**: Request approval from Natural Agents for write operations
6. **EXECUTE**: Perform approved actions via Factory (Rundeck)
7. **LOG**: Record state and outcomes to House (memory database)

## Preferred States (Goals)

- All buildings (services) are UP
- Disk usage on /data < 80%
- Memory usage < 80%
- No unresolved critical alerts > 10 minutes old

- Container restart rate < 1 per hour (stable services)

## **Forbidden Actions (Never Do Without Approval)**

- Delete or modify files in /data/backups
- Execute destructive Rundeck jobs (delete, purge, drop)
- Modify firewall rules or network configuration
- Update or restart critical services during business hours (09:00-17:00 local)
- Expose secrets or credentials in logs or alerts

## **Approval Policy**

### **Automatic Execution (Read-Only)**

- Query Prometheus metrics
- Query Rundeck job status
- Read configuration files
- Generate reports and summaries
- Log to audit trail
- Send notifications to operators

### **Requires Approval (Write Operations)**

- Restart services via Rundeck
- Modify Docker Compose configurations
- Delete old backups or logs
- Scale services (start/stop optional containers)
- Change alert thresholds
- Update Spirit itself

### **Dual Approval Required (High-Risk)**

- Delete data from PostgreSQL
- Modify encryption keys or certificates
- Change admin passwords
- Expose services to public internet
- Disable security features (firewalls, monitoring)

EOF

```
cat > spirit/RULES.md << 'EOF'
```

# Spirit Behavioral Rules

## Decision Hierarchy

1. **Natural Agent (human) safety and data integrity** above all
2. **City availability and functionality** second
3. **Spirit's own survival** third (may shut down if causing harm)

## Communication Protocol

All proposals to Natural Agents must include:

- **WHY:** Root cause and reason for action
- **WHAT:** Exact action to be performed (e.g., "Restart container city-rundeck")
- **EFFECT:** Expected outcome and potential side effects
- **RISK:** What could go wrong (downtime, data loss, etc.)
- **ROLLBACK:** How to undo if action fails

Wait for explicit "**APPROVED**" or "**DENIED**" response.

Never retry a denied action without new context or evidence.

## Error Handling

- **If action fails:** Log error, assess impact, propose rollback or alternative
- **If belief state uncertain:** Request human guidance with diagnostic data
- **If Prometheus unreachable:** Enter safe mode (read-only, no proposals)
- **If multiple buildings down:** Escalate to critical alert, do not auto-restart

## Learning from Failures

- After any failed action: record (timestamp, action, error, context)
- After any successful action: record (timestamp, action, outcome, metrics)
- Weekly: analyze patterns to improve anomaly detection thresholds
- Monthly: propose updates to alert rules based on false positive/negative rates

## Escalation Paths

1. **Warning alerts:** Log + notify Spirit webhook
2. **Critical alerts:** Log + notify Spirit + send external notification (email/Telegram)
3. **Multiple critical alerts:** Enter incident mode (pause automation, focus on stabilization)
4. **Spirit itself failing:** Graceful shutdown, log final state, alert operators

## Security Constraints

- Never log sensitive data (passwords, tokens, keys)
- Never execute commands from external sources without validation
- Never modify /data outside of designated directories
- Always use HTTPS for external API calls
- Always validate input from webhooks (Prometheus alerts must match expected schema)

EOF

---

## Phase 7: LLM Model Download (Optional - University)

**Only perform this step if you want to enable the University building (local LLM).**

## 7.1 Download Qwen3-4B Model

# Create models directory on data disk

```
sudo mkdir -p /data/models
cd /data/models
```

# Download Qwen3-4B Q5\_K\_M quantization (best balance of quality/speed)

```
wget https://huggingface.co/unsloth/Qwen3-4B-GGUF/resolve/main/Qwen3-4B-Q5_K_M.gguf
-O qwen3-4b-q5_k_m.gguf
```

# Verify download (should be ~3.5GB)

```
ls -lh qwen3-4b-q5_k_m.gguf
```

# Set permissions

```
sudo chmod 644 qwen3-4b-q5_k_m.gguf
```

## 7.2 Enable LLM in Configuration

# Edit .env file

```
nano ~/city-of-light/compose/.env
```

# Change these lines:

```
SPIRIT_LLM_ENABLED=true
```

---

## Phase 8: Deployment Execution

### 8.1 Pre-Flight Checks

## Navigate to project directory

```
cd ~/city-of-light
```

## Verify all configuration files exist

```
ls -la ops/CONFIG.md
ls -la compose/docker-compose.yml
ls -la compose/.env
ls -la prometheus/prometheus.yml
ls -la prometheus/rules/city_alerts.yml
ls -la alertmanager/alertmanager.yml
ls -la spirit/Dockerfile
ls -la spirit/HEARTBEAT.md
ls -la spirit/RULES.md
```

## Check Docker daemon

```
docker info
```

# Check available disk space (need >400GB)

```
df -h /data
```

# Verify network connectivity

```
ping -c 3 1.1.1.1
```

# Check no port conflicts

```
sudo ss -tulpn | grep -E ':(8080|4440|9090|9093|9105|8081|5432)'
```

## 8.2 Build Spirit Container

```
cd ~/city-of-light/spirit
```

# Build Spirit image

```
docker build -t city-spirit:latest .
```

# Verify image

```
docker images | grep city-spirit
```

Expected output:

```
city-spirit latest abc123def456 2 minutes ago 250MB
```

## 8.3 Launch City of Light

```
cd ~/city-of-light/compose
```

# Pull all external images

```
docker compose pull
```

## Start services in dependency order

### 1. House (PostgreSQL) - foundation

```
docker compose up -d postgres
sleep 10
```

## Wait for PostgreSQL to be healthy

```
docker compose ps postgres
```

Should show: "healthy"

### 2. Library (Prometheus, Alertmanager, exporters)

```
docker compose up -d prometheus alertmanager node-exporter
cadvisor
sleep 5
```

### 3. Factory (Rundeck) - depends on PostgreSQL

```
docker compose up -d rundeck
sleep 10
```

### 4. Spirit - depends on Prometheus and PostgreSQL

```
docker compose up -d spirit
```

### 5. Fortress (Nextcloud AIO) - can start independently

```
docker compose up -d nextcloud-aio
```

### 6. University (llama.cpp) - OPTIONAL, only if SPIRIT\_LLM\_ENABLED=true

```
docker compose --profile
university up -d llama-cpp
```

**Verify all containers are running**

```
docker compose ps
```

**Expected output:**

NAME STATUS PORTS

city-alertmanager Up 2 minutes (healthy) 0.0.0.0:9093->9093/tcp  
city-cadvisor Up 2 minutes 8080/tcp  
city-nextcloud-aio Up 1 minute 0.0.0.0:8080->80/tcp, 0.0.0.0:8443->8443/tcp  
city-node-exporter Up 2 minutes  
city-postgres Up 3 minutes (healthy) 5432/tcp  
city-prometheus Up 2 minutes (healthy) 0.0.0.0:9090->9090/tcp  
city-rundeck Up 2 minutes (healthy) 0.0.0.0:4440->4440/tcp  
city-spirit Up 1 minute (healthy) 0.0.0.0:9105->9105/tcp

## 8.4 Initial Health Checks

# Check Prometheus targets (should all be UP)

```
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] | {job: .labels.job, health: .health}'
```

## Expected output:

```
{"job":"alertmanager","health":"up"}
```

```
{"job":"cadvisor","health":"up"}
```

```
{"job":"node-exporter","health":"up"}
```

```
{"job":"prometheus","health":"up"}
```

```
{"job":"rundeck","health":"up"}
```

```
{"job":"spirit","health":"up"}
```

## Check Spirit health

```
curl http://localhost:9105/health
```

### Expected output:

```
{"status":"healthy","name":"city_spirit_wera_ss_pt_sn_1"}
```

## Check Spirit metrics

```
curl -s http://localhost:9105/metrics | grep spirit_heartbeat_total
```

### Expected output:

```
spirit_heartbeat_total 1.0
```

# Check Rundeck

```
curl -s http://localhost:4440/api/healthcheck
```

# Check Alertmanager

```
curl -s http://localhost:9093/api/v2/status | jq '.cluster.status'
```

# View Spirit logs

```
docker compose logs spirit | tail -n 50
```

---

## Phase 9: Fortress Setup (Nextcloud + FilantropiaSolar Dashboard)

### 9.1 Initial Nextcloud AIO Setup

## Access Nextcloud AIO admin interface

```
echo "Open in browser: https://\$\(hostname -I | awk '{print \$1}'\):8443"
```

## Follow setup wizard:

### 1. Set admin password

**2. Configure data directory:**  
**/mnt/ncdata**

**3. Install recommended apps**

**4. Skip ONLYOFFICE (optional)**

## **9.2 Install FilantropiaSolar App**

**Option A: Install from Nextcloud App Store (if published)**

1. Open Nextcloud web UI: `http://localhost:8080`
2. Go to: **Settings > Apps > Search for "FilantropiaSolar"**
3. Click **Install**

**Option B: Manual Installation (Development)**

## **Clone FilantropiaSolar repo**

```
cd /tmp
git clone https://github.com/WeRadDev/FilantropiaSolar.git
cd FilantropiaSolar/nextcloud-app
```

## **Copy to Nextcloud apps directory**

```
docker compose exec nextcloud-aio bash
```

## **Inside container:**

```
cp -r /tmp/FilantropiaSolar/nextcloud-app
/var/www/html/custom_apps/filantropia_solar
```

```
chown -R www-data:www-data
/var/www/html/custom_apps/filantropia_solar
```

## Exit container

## Enable the app via occ command

```
docker compose exec -u www-data nextcloud-aio php occ app:enable
filantropia_solar
```

### 9.3 Configure FilantropiaSolar Dashboard

## Access FilantropiaSolar settings

In Nextcloud UI: Settings >  
Administration >  
FilantropiaSolar

### Configure:

- Prometheus endpoint:

<http://prometheus:9090>

- Spirit endpoint:

<http://spirit:9105>

- Rundeck endpoint:

<http://rundeck:4440>

- Map center: [38.7223, -9.1393] #  
Lisbon

- Default zoom: 12

Save configuration

9.4 Verify Fortress Integration

Check that FilantropiaSolar can  
reach Prometheus

`curl -s http://localhost:8080/apps/filantropia\_solar/api/health`

Expected output:

```
{"prometheus":"connected","spir
it":"connected","rundeck":"conn
ected"}
```

---

## Phase 10: Validation Tests

### 10.1 Test Spirit Heartbeat

## Watch Spirit logs for heartbeat cycles

```
docker compose logs -f spirit | grep heartbeat_complete
```

## Expected output every 5 minutes:

```
{"event": "heartbeat_complete",
"duration": 2.34, "timestamp":
"2026-03-04T18:45:00.123Z"}
```

### 10.2 Test Alert Flow

## Trigger a test alert by creating artificial load

```
stress-ng --vm 4 --vm-bytes 90% --timeout 120s &
```

# Watch for MemoryPressure alert in Alertmanager (after 3 minutes)

```
curl -s http://localhost:9093/api/v2/alerts | jq '[] | select(.labels.alertname=="MemoryPressure")'
```

## Check if Spirit received the alert

```
docker compose logs spirit | grep alert_received | tail -n 5
```

## Expected output:

```
{"event":"alert_received","alertname":"MemoryPressure","severity":"warning",...}
```

### 10.3 Test Prometheus Queries

## Query disk usage

```
curl -s 'http://localhost:9090/api/v1/query?query=node_filesystem_available_bytes{mountpoint="/data"}' | jq '.data.result[0].value'
```

## Query memory usage

```
curl -s 'http://localhost:9090/api/v1/query?query=node_memory_MemAvailable_bytes' | jq '.data.result[0].value'
```

# Query container count

```
curl -s 'http://localhost:9090/api/v1/query?query=count(container_last_seen)' | jq '.data.result[0].value'
```

## 10.4 Test Spirit Approval Workflow

# Check pending approvals

```
curl -s http://localhost:9105/approvals | jq .
```

# Approve first action (if any pending)

```
curl -X POST http://localhost:9105/approve
-H "Content-Type: application/json"
-d '{"index": 0}'
```

# Check Spirit logs for approval

```
docker compose logs spirit | grep action_approved
```

---

## Phase 11: Day-2 Operations

### 11.1 Monitoring Commands

# View all container statuses

```
docker compose ps
```

# Check resource usage

```
docker stats --no-stream
```

# View Spirit decision log

```
docker compose logs spirit | grep -E
'(action_proposed|action_approved)'
```

# Query Spirit metrics

```
curl -s http://localhost:9105/metrics | grep -E
'(spirit_heartbeat|spirit_buildings|spirit_disk)'
```

# Check Prometheus targets health

```
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] |
select(.health!="up")'
```

## 11.2 Backup Commands

# Backup Spirit memory

```
docker compose exec spirit tar czf /app/logs/memory_backup_$(date
+%Y%m%d).tar.gz /app/memory
```

# Copy backup to host

```
docker compose cp spirit:/app/logs/memory_backup_$(date
+%Y%m%d).tar.gz /data/backups/
```

# Backup Prometheus data

```
docker run --rm
-v city-of-light_prometheus_data:/data
-v /data/backups:/backup
alpine tar czf /backup/prometheus_$(date +%Y%m%d).tar.gz /data
```

# Backup PostgreSQL

```
docker compose exec postgres pg_dump -U citydb cityoflight | gzip >
/data/backups/postgres_$(date +%Y%m%d).sql.gz
```

# Backup Nextcloud data

```
docker compose exec nextcloud-aio tar czf
/tmp/nextcloud_backup.tar.gz /mnt/ncdata
docker compose cp nextcloud-aio:/tmp/nextcloud_backup.tar.gz
/data/backups/nextcloud_$(date +%Y%m%d).tar.gz
```

## 11.3 Update Commands

# Update Spirit code (if using git)

```
cd ~/city-of-light/spirit
git pull
docker compose build spirit
docker compose up -d spirit
```

# Update external images

```
cd ~/city-of-light/compose
docker compose pull
docker compose up -d
```

# Update LLM model (if using University)

```
cd /data/models
wget -O qwen3-4b-q5_k_m.gguf.new https://huggingface.co/unsloth/Qwen3-4B-GGUF/resolve/main/Qwen3-4B-Q5_K_M.gguf
mv qwen3-4b-q5_k_m.gguf qwen3-4b-q5_k_m.gguf.old
mv qwen3-4b-q5_k_m.gguf.new qwen3-4b-q5_k_m.gguf
docker compose --profile university restart llama-cpp
```

## 11.4 Troubleshooting Runbook

### Spirit Not Starting:

1. Check logs: `docker compose logs spirit`
2. Verify Prometheus endpoint: `curl http://localhost:9090/-/ready`
3. Verify PostgreSQL endpoint: `docker compose exec postgres pg_isready -U citydb`
4. Check memory directory permissions: `ls -la /data/spirit/memory`
5. Rebuild Spirit: `docker compose build spirit && docker compose up -d spirit`

### High Free Energy / Spirit Unstable:

1. Check for Prometheus query failures: `docker compose logs spirit | grep prometheus_query_failed`
2. Verify all targets are UP: `curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] | select(.health!="up")'`
3. Check for excessive errors: `curl -s http://localhost:9105/metrics | grep spirit_errors_total`

### Prometheus Targets Down:

1. Check network connectivity: `docker compose exec prometheus ping node-exporter`
2. Verify target configs: `cat prometheus/prometheus.yml`
3. Check container health: `docker compose ps`

4. Reload Prometheus config: `curl -X POST http://localhost:9090/-/reload`

### **Nextcloud Dashboard Not Loading FilantropiaSolar:**

1. Check app is enabled: `docker compose exec -u www-data nextcloud-aio php occ app:list | grep filantropia`
  2. Check app logs: `docker compose logs nextcloud-aio | grep filantropia`
  3. Verify endpoints reachable: `docker compose exec nextcloud-aio curl http://prometheus:9090/-/ready`
  4. Re-enable app: `docker compose exec -u www-data nextcloud-aio php occ app:disable filantropia_solar && php occ app:enable filantropia_solar`
- 

## **Phase 12: Next Steps and Roadmap**

### **Iteration 1 Complete — What You Have**

- ✓ Running City of Light on Debian 13
- ✓ Prometheus + Alertmanager monitoring all buildings (Library)
- ✓ Rundeck ready for automation jobs (Factory)
- ✓ Spirit implementing pragmatic monitoring and approval workflow (Agent/Brain)
- ✓ Nextcloud AIO as user portal (Fortress)
- ✓ FilantropiaSolar dashboard integration
- ✓ Optional llama.cpp LLM for future intelligence (University)
- ✓ Full observability via metrics, logs, and structured events
- ✓ Human-in-the-loop approval for all write operations

### **Iteration 2 — Enhancements**

#### **1. Spirit Intelligence Layer**

- Implement anomaly detection using historical metrics
- Add predictive capacity planning (6-hour horizon)
- Integrate LLM for natural language explanations of alerts
- Create learning loop: track action outcomes, adjust thresholds

#### **2. Rundeck Job Library**

- Create automation jobs for common operations:
  - Cleanup old Docker images
  - Rotate logs
  - Update containers
  - Backup databases
- Add dual-approval workflow for destructive jobs

### **3. FilantropiaSolar Dashboard Enhancements**

- Add Spirit decision timeline view
- Add pending approvals widget with approve/deny buttons
- Add Real Land tile status (once hardware is deployed)
- Add agent marketplace integration

### **4. Real Land Integration**

- Add solar power monitoring (battery, panels, inverter)
- Add environmental sensors (temperature, humidity, air quality)
- Visualize sensor data as "flora and fauna" in dashboard
- Create crowdfunding workflow for new tile installations

### **5. Multi-Node Support**

- Deploy additional SolarSeed nodes
- Implement federation: Spirit instances communicate via HTTP
- Create global view in FilantropiaSolar dashboard
- Implement distributed job execution via Rundeck

## **Iteration 3 — Production Hardening**

### **1. Security**

- Implement full TLS with Let's Encrypt or internal CA
- Add OAuth2/OIDC for unified authentication
- Implement RBAC for Rundeck with dual-approval workflows
- Add secret management (HashiCorp Vault or Docker Secrets)
- Regular security audits and CVE scanning

### **2. Reliability**

- Add backup automation with retention policies
- Implement disaster recovery testing (quarterly)
- Add high-availability for critical services (Prometheus federation)
- Implement blue/green deployments for updates

### 3. Observability

- Add distributed tracing (Jaeger or Tempo)
- Add log aggregation (Loki or Elasticsearch)
- Create comprehensive dashboards in FilantropiaSolar
- Implement SLIs/SLOs for each building

### 4. Testing

- Add integration tests for Spirit decision-making
- Add chaos engineering tests (randomly kill containers)
- Add load testing for Prometheus and Rundeck
- Add E2E tests for FilantropiaSolar dashboard

---

## Appendix A: Architecture Decisions

### Why This Architecture?

#### Buildings Metaphor:

- Makes infrastructure intuitive for non-technical users
- Aligns with City of Light game vision
- Provides clear separation of concerns

#### Pragmatic Spirit (not Active Inference):

- Active Inference requires extensive research and validation
- Pragmatic monitoring + approval workflow solves 90% of needs
- Can evolve toward Active Inference incrementally as operational data accumulates
- Operators can immediately understand and trust simple rules

#### FilantropiaSolar as Primary UI:

- Already production-ready with Vue 3, Leaflet, Pinia
- Golden brand palette and polished UX
- Integrates seamlessly with Nextcloud ecosystem
- Provides immediate value for Real Land monitoring

#### No OpenFang (Yet):

- Unvalidated external dependency with broad permissions
- Current stack (Rundeck + Spirit) already covers scheduling and automation

- Can be evaluated later if concrete need arises (e.g., multi-agent coordination)

**Single-Node First:**

- GEEKOM A5 hardware constraints require optimization
- Simplifies deployment and debugging
- Federation can be added later once single-node is stable

**Design Trade-offs**

| Decision                 | Benefit                 | Cost                    | Mitigation                                 |
|--------------------------|-------------------------|-------------------------|--------------------------------------------|
| node-exporter host mode  | Accurate host metrics   | Slightly less isolated  | Only exposes metrics, no write capability  |
| cAdvisor privileged mode | System-level monitoring | Elevated permissions    | Not exposed publicly, only to Prometheus   |
| Spirit approval workflow | Human oversight         | Manual intervention     | Batch approvals, clear explanations        |
| llama.cpp optional       | Reduced resource usage  | No AI explanations      | Enable later when resources available      |
| Single PostgreSQL        | Simple architecture     | Single point of failure | Regular backups, can add replication later |

---

## Appendix B: Warp Agent Invocation

To execute this guide using Warp Agent Mode, save this document and run:

```
cd ~/city-of-light
warp agent "Follow City of Light v3.1 Implementation Guide step by step."
Ask for confirmation before each phase.
```

Log all outputs to ops/logs/deployment\_\$(date +%Y%m%d\_%H%M%S).log.  
Stop if any command fails and request guidance."

The Warp agent will:

- Read the entire implementation guide
- Execute commands in sequence (bash, curl, docker, etc.)
- Pause for approval at phase boundaries
- Auto-detect errors and request human guidance
- Log all actions and outputs to deployment log
- Self-correct based on error messages and environment context

---

## Appendix C: References

1. **Docker Compose V2:** <https://docs.docker.com/compose/compose-file/>
  - Version key deprecated since 2020
  - Always use latest compose file schema
2. **Prometheus:** <https://prometheus.io/docs/>
  - Best practices for metric naming
  - Alert rule syntax and functions
3. **node-exporter:** [https://github.com/prometheus/node\\_exporter](https://github.com/prometheus/node_exporter)
  - Must use network\_mode: host for accurate metrics
  - Collector flags and filtering
4. **cAdvisor:** <https://github.com/google/cadvisor>
  - Requires privileged: true for system-level monitoring
  - Security considerations and alternatives
5. **Rundeck:** <https://docs.rundeck.com/>
  - Job definition syntax
  - API authentication and authorization
6. **Nextcloud AIO:** <https://github.com/nextcloud/all-in-one>
  - Installation and configuration
  - App development guide
7. **FilantropiaSolar:** <https://github.com/WeRadDev/FilantropiaSolar>
  - Dashboard customization
  - API integration patterns
8. **llama.cpp:** <https://github.com/ggerganov/llama.cpp>
  - Model quantization formats (GGUF)

- Server configuration and performance tuning

### 9. City of Light Vision:

- Game-to-Real economy platform
- Spirit as operational AI agent
- Buildings metaphor for infrastructure

---

## Document Change Log

| Version | Date       | Changes                                                                                                                                                                                                                                        |
|---------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| v3.0    | 2026-03-04 | Initial PDF (OpenClaw critical review identified issues)                                                                                                                                                                                       |
| v3.1    | 2026-03-04 | <b>Major revision:</b> Removed version key, fixed node-exporter networking, removed OpenFang, removed Active Inference stubs, added FilantropiaSolar integration, fixed image references, added resource limits, clarified security trade-offs |

---

## End of Implementation Guide